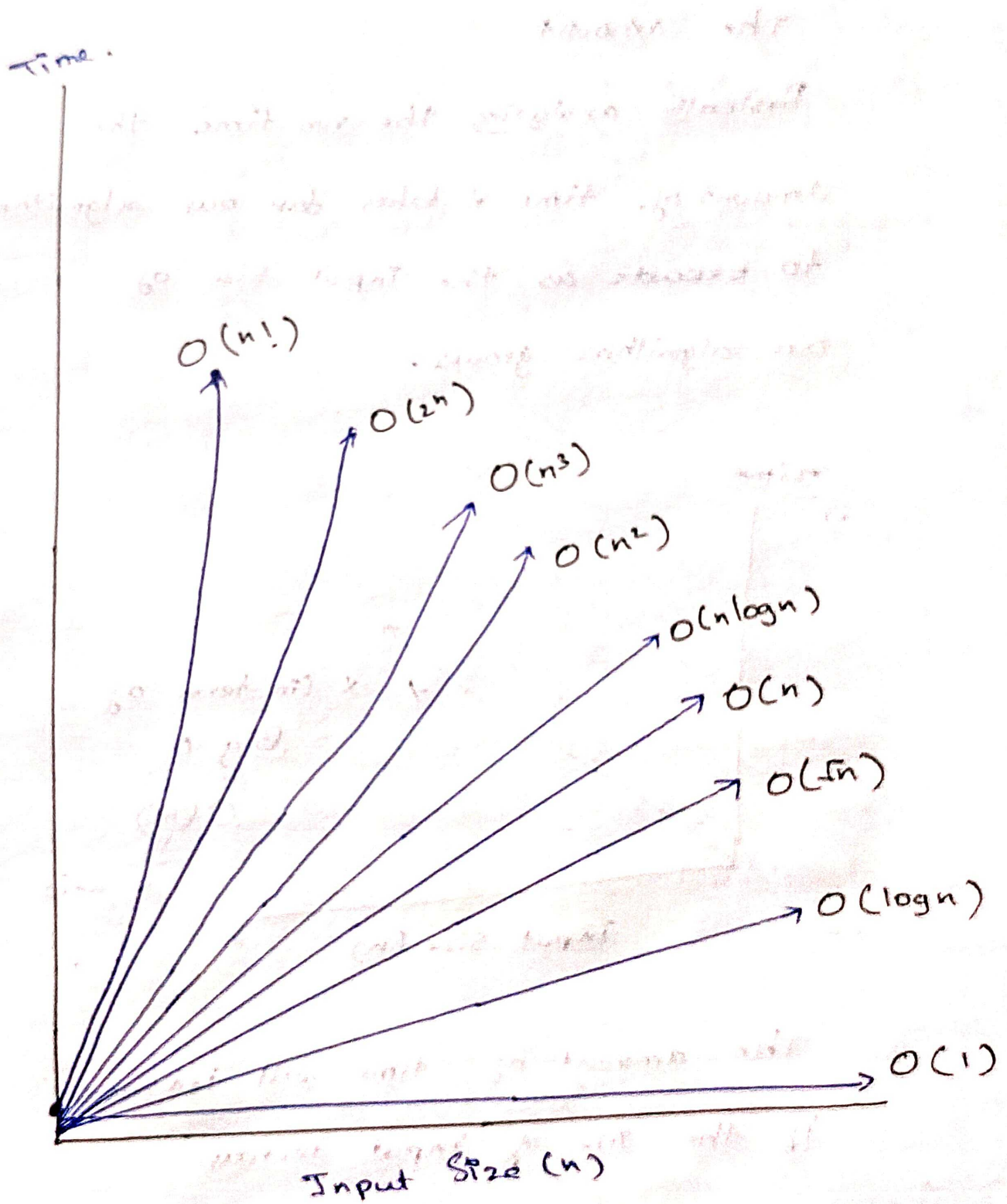


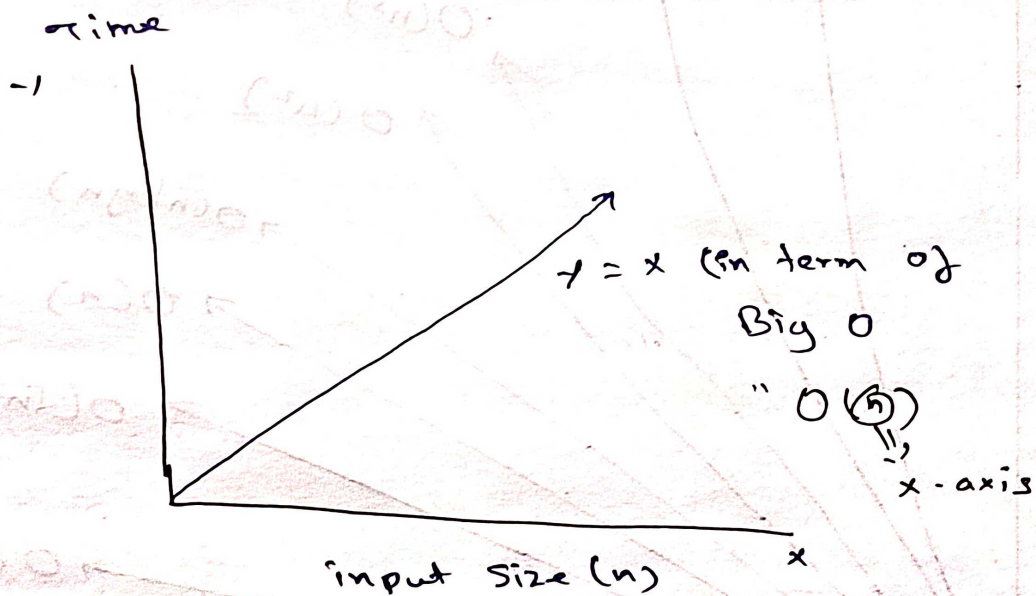
Big "O"



Big O?

The amount

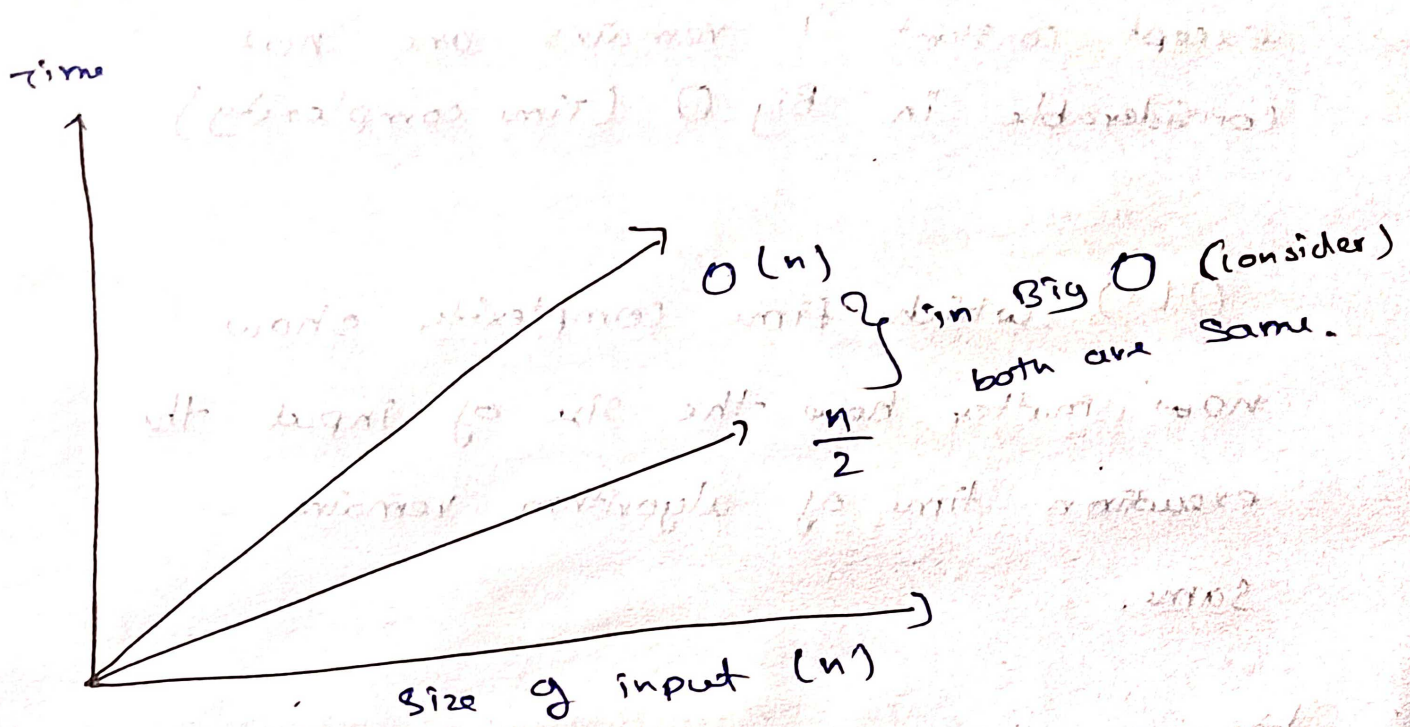
Basically analyzing the run time the amount of time it takes for our algorithm to execute as the input size of our algorithm grows.



The amount of time will increase

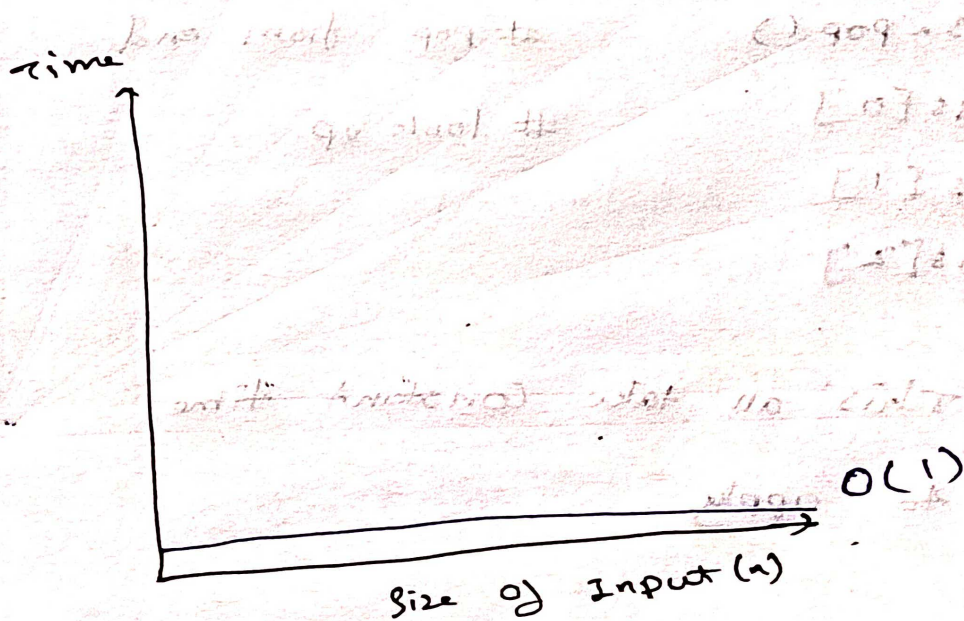
if the size of input increases.

which basically a "linear time stop"



in Big O we really don't care about constant. only variable n is consider.

1) $O(1)$ (an efficient algorithm)



except constant "1" remains are not considerable in Big O (time complexity)

$O(1)$ which time complexity show now matter how the size of input the execution time of algorithm remains same.

for example:

Array.

nums = [1, 2, 3]

nums.append(n) # push to end

nums.pop() # pop from end

nums[0] # look up

nums[1]

nums[2]

This all take constant time
to apply

HashMap / set.

hashMap = {}

hashMap["key"] = 10 # insert

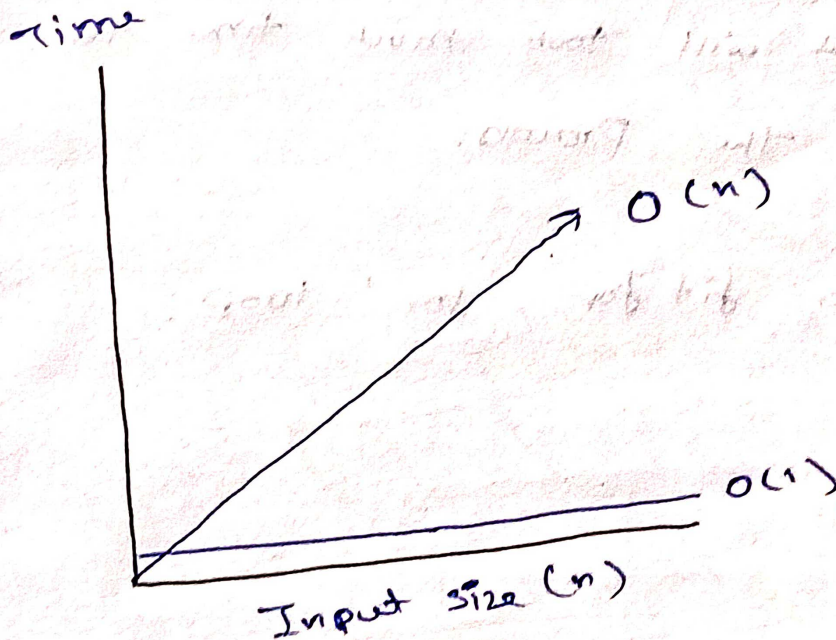
Print("key" in hashMap) # lookup

Print(hashMap["key"]) # lookup

hashMap.pop("key") # remove.

Same, this operation in both hash and set took constant time to apply.

2) $O(n)$



The input size grows our time is going to grow proportionately.

for example :

nums = [1, 2, 3]

sum(nums) # sum of array

for n in nums: # looping

Print(n)

both take same time complexity

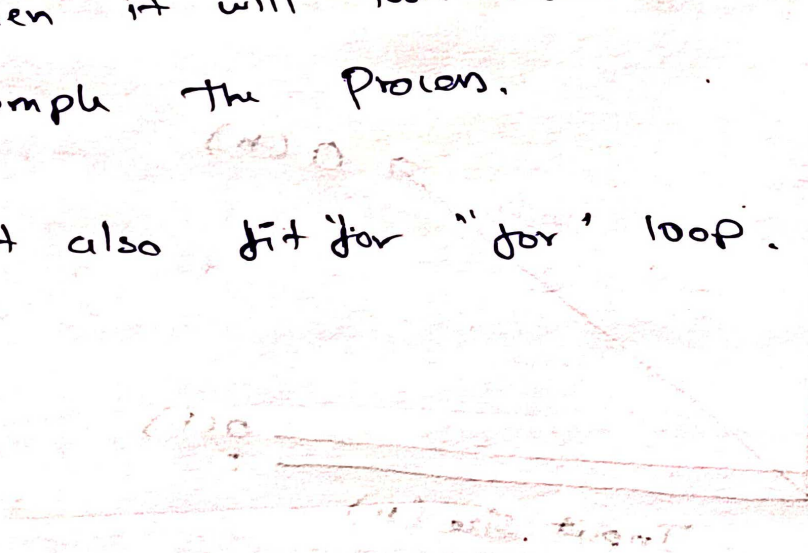
$O(n)$ because the num & size of
input increase the time also increase

'never' 3 item if it has 100 item

then it will took much time to

comple the process.

it also fit for "for" loop.



nums.insert(1, 100) # insert middle

nums.remove(100) # remove middle

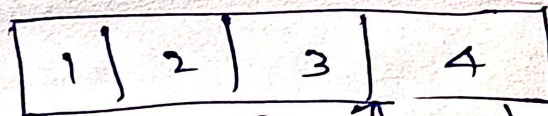
Print(100 in nums) # search.

for this (insert).

let's assume.



we have to insert in the middle
whether it could be in position 1
or 3 or 4.

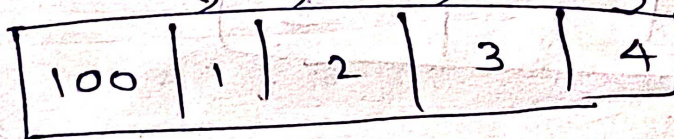


100



(or)

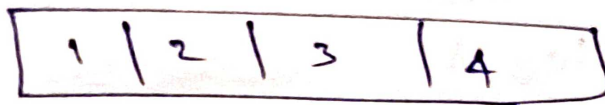
100



we have to shift the position
of other element.

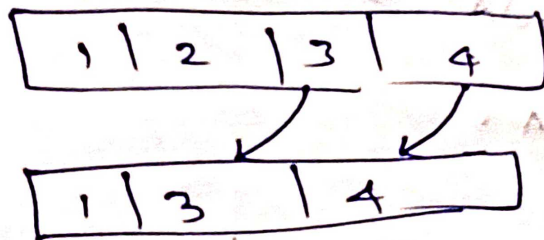
remove.

Same as the insert the remove also have to shift the position after the process.



remove (2)

nums.remove(2)



search.

100 in a array



for searching we have take element by index and compare.

always to worst case scenario if searching
element present in the list or whether not
present in array we have to search each and
every element in array.

```
import heapq
```

```
heapq.heapify(nums) # build heap.
```

This also and time operation is $O(n)$

sometimes even nested loops can be $O(n)$

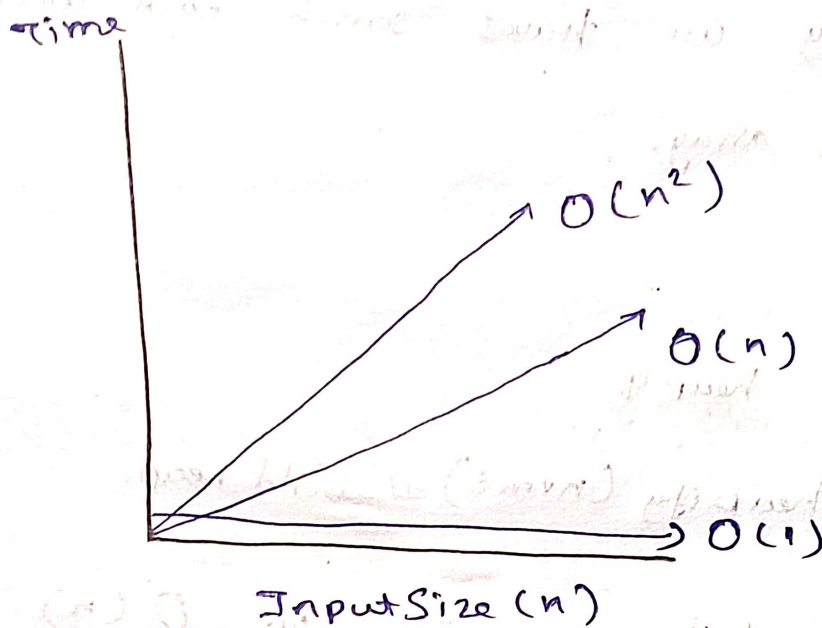
(eg monotonic stack or sliding window)

Note:

nested loops are not always n^2 time
complexity.

linear time means the amount of time
will increase if the input size increases.

3) $O(n^2)$



for example.

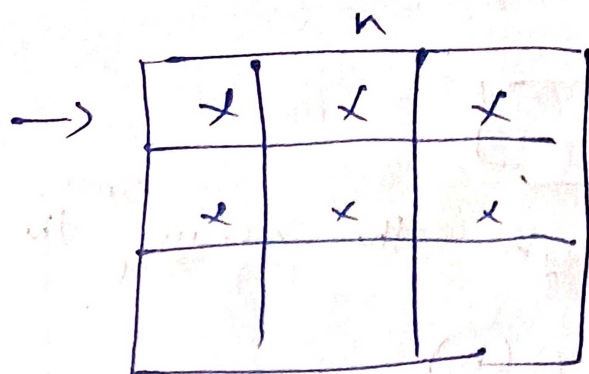
Traverse a square grid

```
nums = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
for i in range(len(nums)):
```

```
    for j in range(len(nums[i])):
```

```
        Print(nums[i][j])
```

for example consider a 3×3 or 2×2 grid.

To iterate the element in array we would need nested loop.

The outer loop for row and inner loop for item.

Get every pair of element in array.

```
nums = [1, 2, 3]
```

```
for i in range(len(nums)):
```

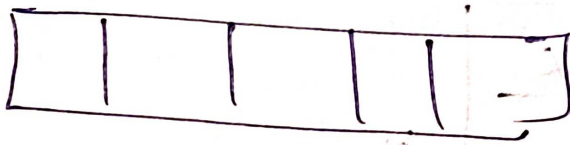
```
    for j in range(i+1, len(nums)):
```

```
        print(nums[i], nums[j])
```

Insertion Sort

(insert in middle n times $\rightarrow n^2$)

take an array.



if we iterate through the array ^{once} the time complexity is $O(n)$

if we iterate through the array twice the time complexity is $O(2n)$

for 3 the time complexity is $O(3n)$

if n^2 the time complexity is

$$O(n \cdot n) \Rightarrow O(n^2)$$

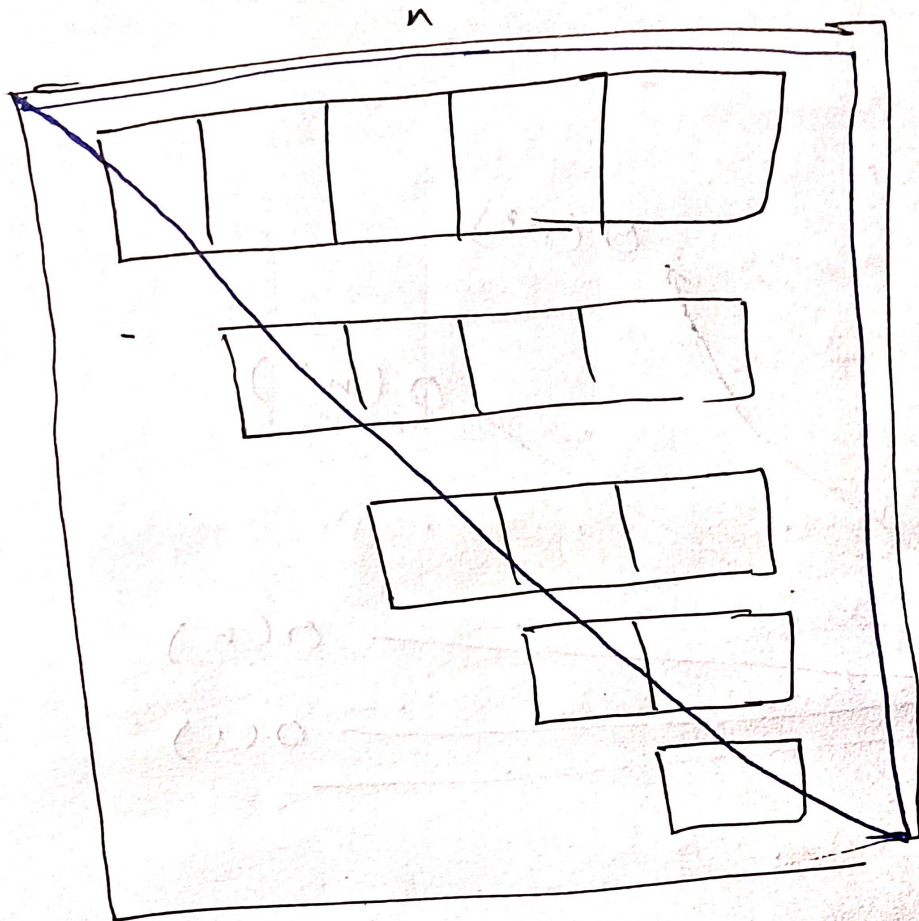
$$O(1 \times n) = O(n)$$

$$O(2 \times n) = O(2n)$$

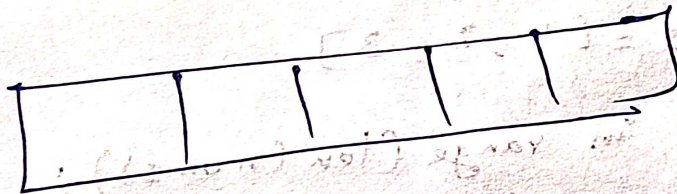
$$O(3 \times n) = O(3n)$$

as we learned we simply neglect the constant. so $O(n)$ not in power.

so $O(n^2)$ same $O(n^2)$



above is example of array using nested loops.



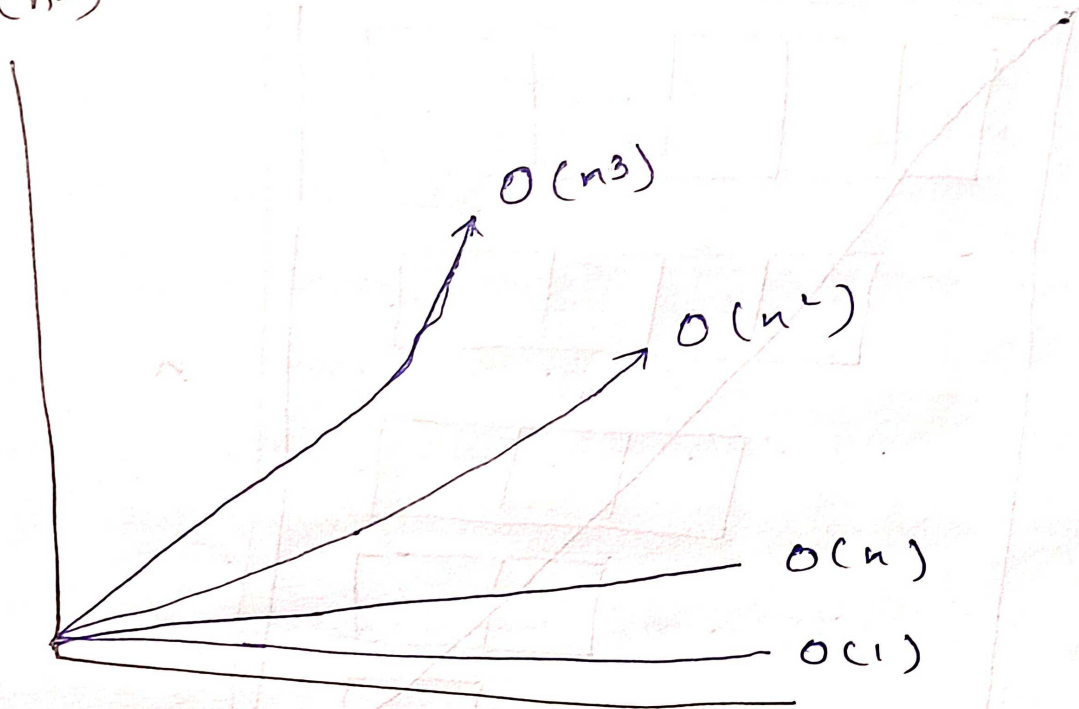
So time complexity is

$$O\left(\leq \frac{n^2}{2}\right) \quad (\text{or})$$

$\frac{n^2}{2} \Rightarrow$ neglect the "constant"

$$\text{So } O(n^2)$$

4) $O(n^3)$



for example.

Get every triplet of elements in array.

`nums = [1, 2, 3]`

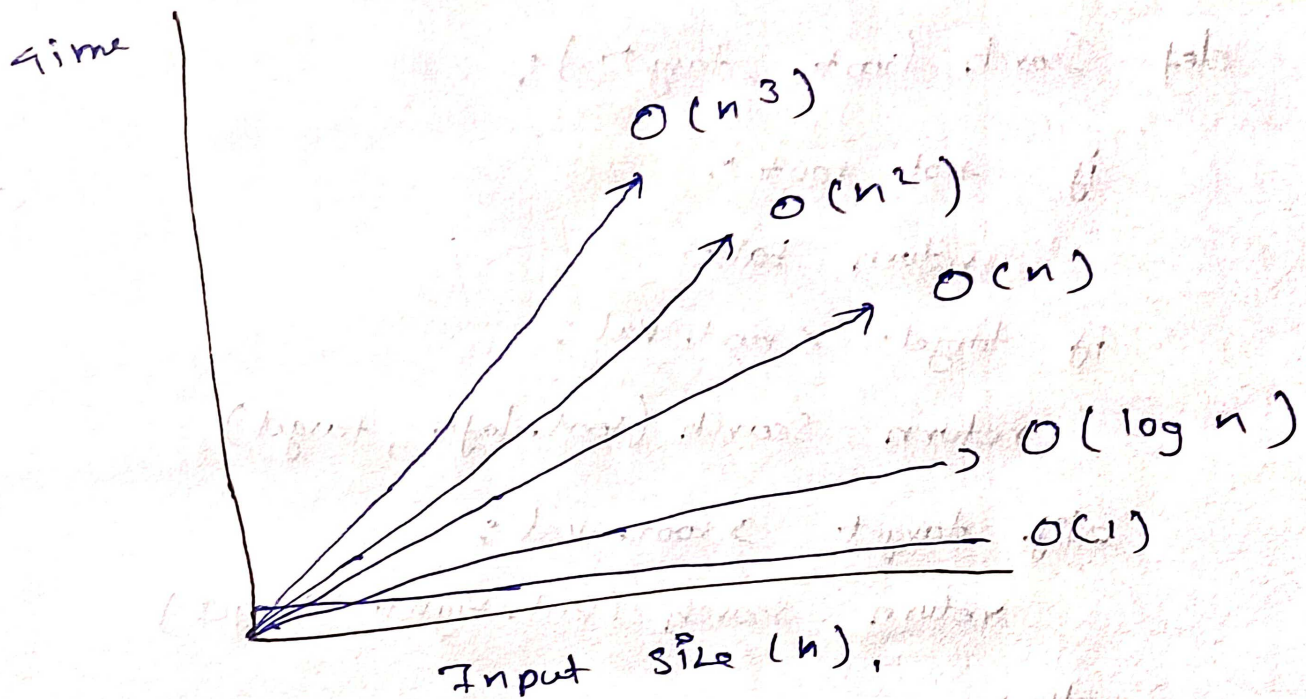
`for i in range(len(nums)):`

`for j in range(i+1, len(nums)):`

`for k in range(j+1, len(nums)):`

`Print(nums[i], nums[j], nums[k])`

5) $O(\log n)$



ex:

The example for " $\log n$ " is

Binary Search.

nums = [1, 2, 3, 4, 5]

target = 6

l = 0, len(nums) - 1

while l <= r:

m = (l + r) // 2

if target < nums[m]:

r = m - 1

elif target > nums[m]:

l = m + 1

else:

print(m)

break

Binary Search on BST.

```
def Search (root, target):
```

```
    if not root:
```

```
        return False
```

```
    if target < root.val:
```

```
        return Search (root.left, target)
```

```
    elif target > root.val:
```

```
        return Search (root.right, target)
```

```
    else:
```

```
        return True.
```

Heap push and Pop.

```
import heapq
```

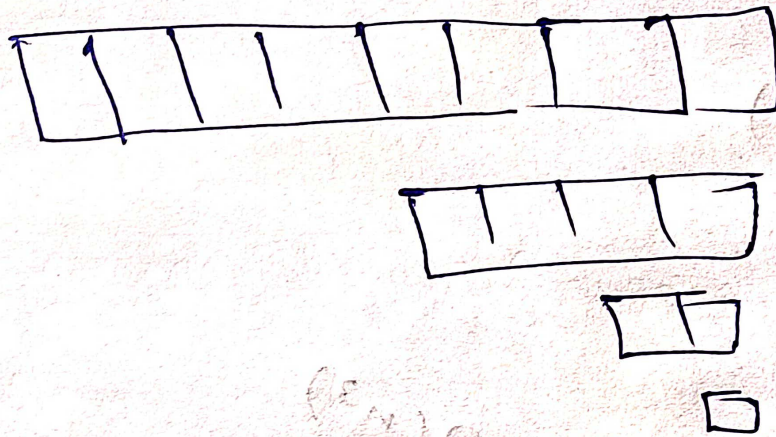
```
minHeap = []
```

```
heapq.heappush (minHeap, 5)
```

```
heapq.heappop (minHeap)
```


On an array the reason Binary search is $O(\log n)$ is because the Binary search eliminates half of the portion then execute. Each time we execute it reduce by 2 which is divided by 2.

for ex:



it reduce the half of the portion. So,

basically $\boxed{n/2 = 1}$ how many time we can cut the n by 2 equal to n .

or

$$1 \times 2 \times 2 = n$$

how many time we multiply 2 to equal to n .

$$2^x = n$$

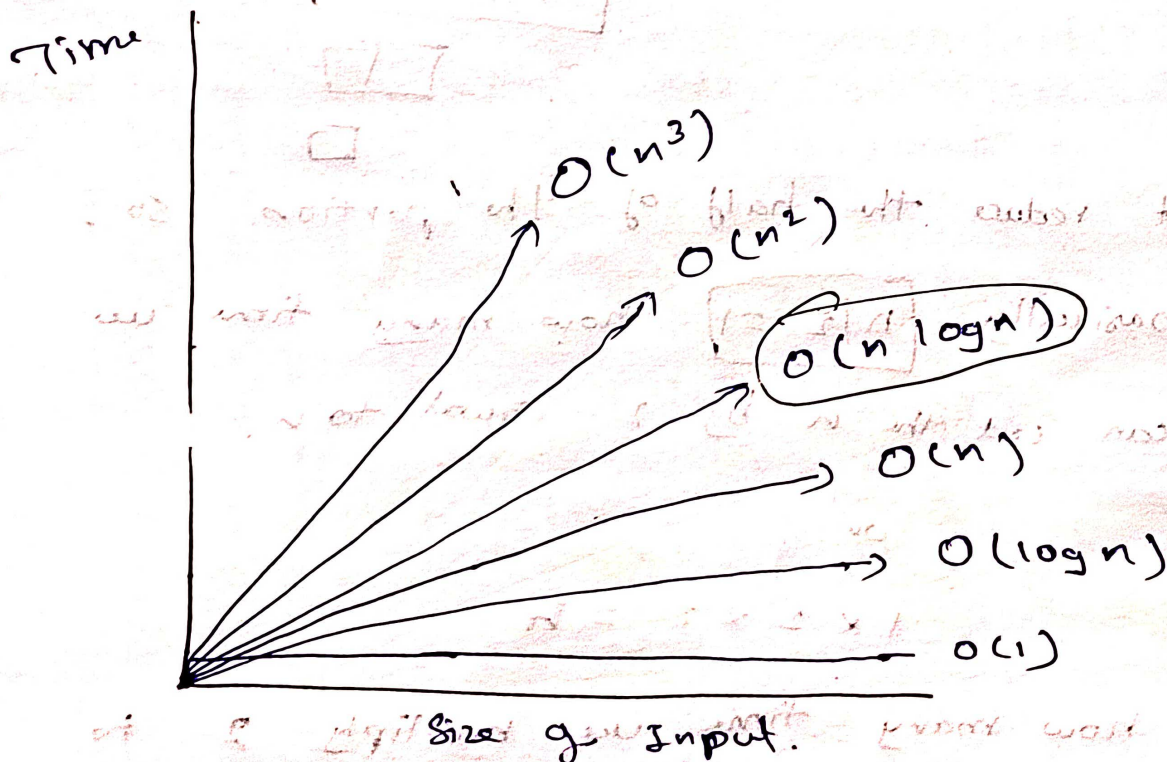
How do we solve mathematically

$$\log_2 2^x = \log_2 n$$

$$x = \log_2 n \Rightarrow n = \frac{\log n}{\log 2}$$

$$O(\log n)$$

b) $O(n \log n)$



The $O(n \log n)$ is marginally less efficient compared to $O(n)$

but it more efficient compare to $O(n^2)$.

ex:

Heap Sort.

```
import heapq
```

```
nums = [1, 2, 3, 4, 5]
```

```
heapq.heapify(nums) #  $O(n)$ 
```

```
while nums:
```

```
    heapq.heappop(nums) #  $O(\log n)$ 
```

Merge Sort

(and most built-in sorting functions)

while True

heappop $\Rightarrow O(\log n)$ and for each so

$O(n)$ before that we have to build

heap so,

$$O(n + n \log n)$$

This should be larger term

so only we consider

$$O(n \log n)$$

to built heap $\Rightarrow O(n)$

to do n $\Rightarrow O(n)$

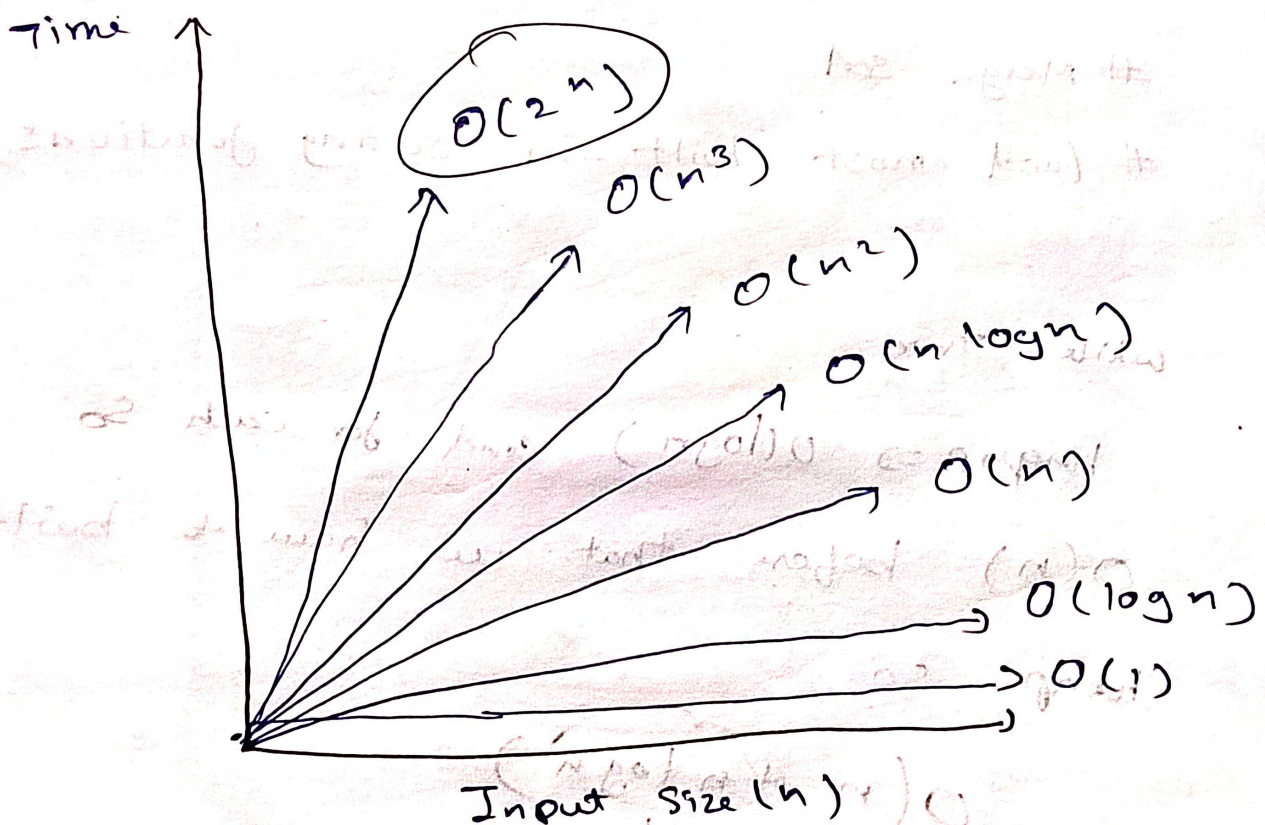
for pop $\Rightarrow O(\log n)$

So,

$$O(n + n \log n)$$

$$O(n \log n)$$

7) $O(2^n)$



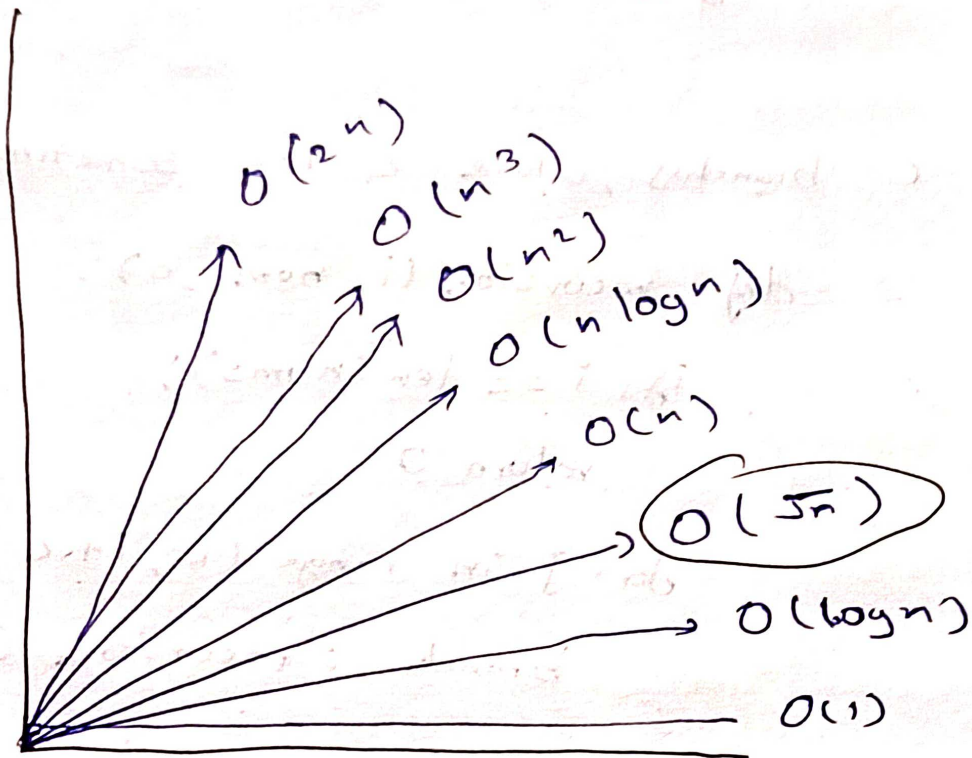
non polynomial run time

$$O(2^n)$$

$$2 \times 2 \times 2 = n \text{ so, } O(2^n)$$

$$3 \times 3 \times 3 \times 3 = n \text{ so, } O(3^n)$$

$$3) O(\sqrt{n})$$



Ex:

Get all factors of n

```
import math
```

```
n = 12
```

```
factors = set()
```

```
for i in range(1, int(math.sqrt(n)) + 1):
```

```
    if n % i == 0:
```

```
        factors.add(i)
```

```
        factors.add(n // i)
```

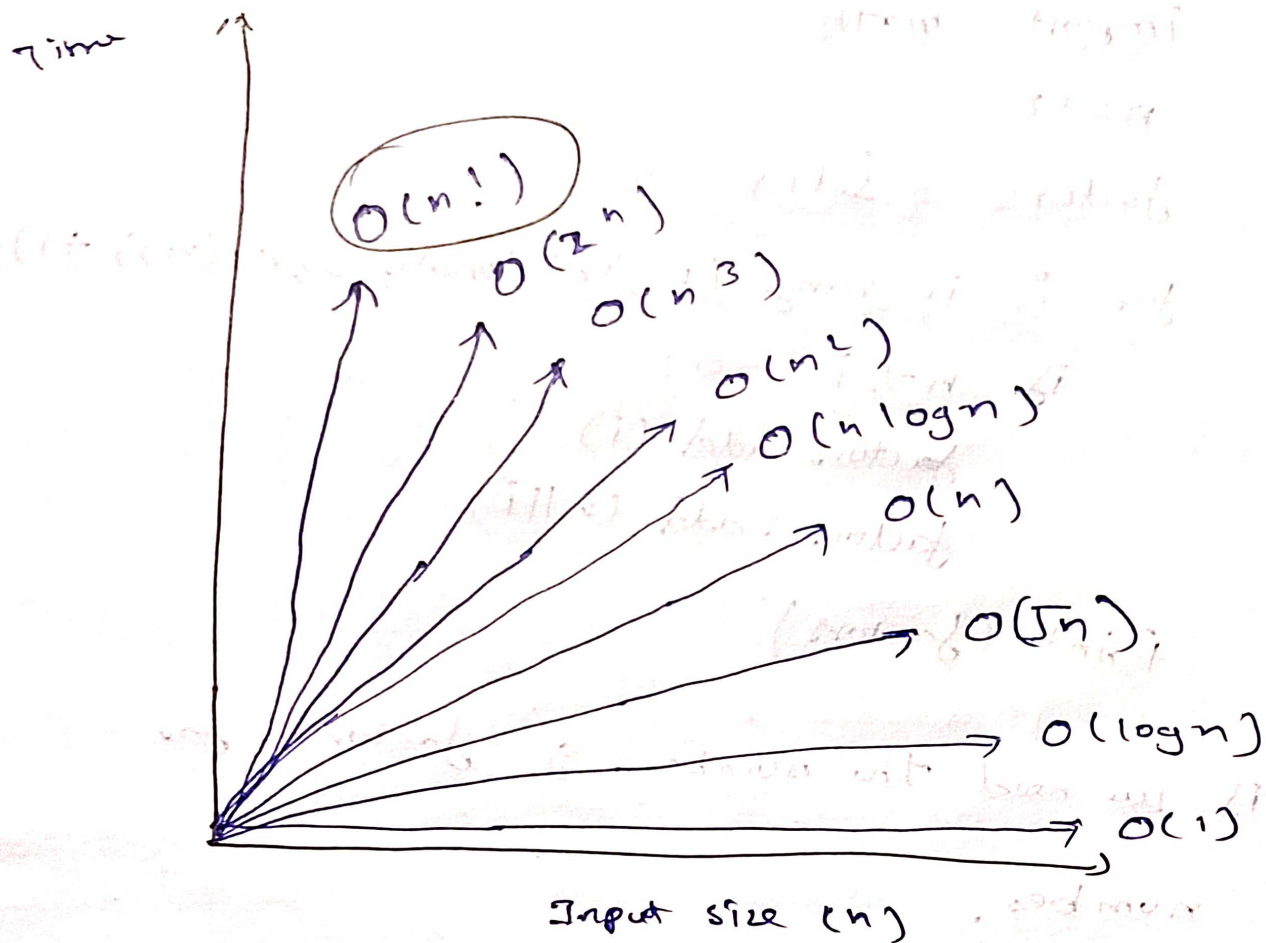
```
print(factors)
```

If we need the number of factors for a number.

it can produce some duplicates so we add into set. we it took constant time.

Math Problem.

9) $O(n!)$



$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

This is bigger than $O(2^n)$ because

$$2^5 = 2 \times 2 \times 2 \times 2 \times 2$$

The base of factorial is higher but for $O(2^n)$ it's constant.

ex:

Permutation

Travelling Salesman Problem.